

쌍거울의 무덤 — 팀원 롤 기술서

다른 시대의 과거를 비추는 두 청동 거울로 무덤을 탈출하는 1인칭 웹 퍼즐 게임 (3인 팀)

1. 팀 구성

이름	담당 역할
김두희	기획 · 개발 · 아트 · AI 활용 (공동)
김수곤	기획 · 개발 · 아트 · AI 활용 (공동)
박철오	기획 · 개발 · 아트 · AI 활용 (공동)

세 명의 역할을 기획·개발·아트로 딱 나누지는 않았습니니다. 그날 필요한 기능을 중심으로 일을 나눴고, 서로의 결과물을 플레이하며 보완했습니다. 다음 장에서는 개인별 업무보다 팀이 실제로 기획하고 구현한 내용을 작업 영역별로 정리했습니다.

2. 실제 구현 영역

2-1. 게임 기획 · 퍼즐 설계

- 시간 규칙:** 퍼즐의 출발점은 '과거에서 바꾼 것만 현재에 남는다'는 원칙이었습니다. 여기에 두 거울이 각각 다른 과거를 보여 준다는 설정과, 과거에서는 거울 빛 안에서만 움직일 수 있다는 제약을 더했습니다. 물건은 거울을 직접 통과할 수 없고, 벽 틈 같은 보관 장소에 남겨 두어야 다음 시대에서 찾을 수 있게 했습니다.
- 세 개의 수수께끼:** 첫 번째는 돌무더기에서 찾은 열쇠를 벽 틈에 숨겨 현재로 가져오는 퍼즐입니다. 다음으로 더 먼 과거의 도구를 가까운 과거에 남겨 벽을 열고, 그 결과를 현재에서 확인하게 했습니다. 마지막에는 보물을 벽화에 대어 얻은 세 글자로 가짜 문을 열게 됩니다.
- 유도 방식:** 순서를 억지로 막기보다는 플레이어가 여러 가지를 시도하며 규칙을 눈치채기를 바랐습니다. 따로 안내문을 놓는 대신, 무너진 자리 위의 금 간 천장이나 시대에 따라 다른 모습의 석판처럼 장면 안에 힌트를 놓았습니다.

2-2. 시간 논리 엔진 · 자동 검증 (개발)

- 시간 상태 처리:** 게임의 진행 상태는 하나로 관리합니다. 각 시대의 장면은 이 상태를 기준으로 계산하므로, 과거에서 문을 열거나 벽을 뜯으면 현재 장면도 바로 달라집니다.
- 막힘 검사:** 플레이어가 도달할 수 있는 112가지 진행 상태를 자동으로 둘러보는 검사를 만들었습니다. 어떤 순서로 풀어도 막힘 상태가 되지 않는지 확인했고, 이 과정에서 실제 설계 문제 두 건을 찾아 고쳤습니다.
- 거울 빛 검증:** 거울을 돌렸을 때 빛이 열쇠나 벽돌까지 실제로 닿는지도 수치로 검사했습니다. 상호작용할 공간이 너무 좁지 않은지, 한 각도에서 두 목표가 모두 빛에 들어와 퍼즐이 의도보다 쉬워지지 않는지를 이 검사로 확인했습니다.
- 자동 테스트:** 위 검사는 `npm test` 명령으로 한 번에 실행됩니다. 작업을 합칠 때마다 이 명령을 돌려, 수정한 내용이 기존 규칙을 깨지 않았는지 봤습니다.

2-3. 3D 그래픽 · 아트

- **3D 장면:** Three.js로 두 방과 세 시대를 만들었습니다. 벽·바닥·문·잔해 등에는 Poly Haven의 CC0 PBR 텍스처 7종을 사용했고, 해골(skull003, Jake K-H)과 1인칭 손(Fists, Sean Tarrant) 모델은 Poly Pizza에서 가져와 CC-BY 조건에 맞게 출처를 남겼습니다. 청동 거울에 보이는 과거는 별도의 장면을 실시간으로 다시 그린 것입니다.
- **벽화와 문양:** 벽화·가짜 문·석판·다이얼은 먼저 직접 초안을 잡았습니다. 그중 기존 에셋이 잘 맞는 부분만 가져오고, 아누비스 벽화·상형문자 따·별 천장·글자 다이얼은 코드로 그렸습니다. 새김 무늬는 먼저 흑백으로 높낮이를 그린 뒤 노멀 맵으로 바꿔, 손전등을 비스듬히 비추면 파인 부분에 그늘이 지도록 했습니다.
- **조명:** 현재의 주된 빛은 플레이어의 손전등이고, 과거에서는 거울에서 나오는 빛기둥이 그 역할을 합니다. 빛기둥은 벽과 문틈에 막히면 그 모양대로 잘립니다. 그래서 빛이 비치는 범위와 플레이어가 움직일 수 있는 범위가 자연스럽게 일치합니다.

2-4. 연출 · 사운드

- **도입과 엔딩:** 시작할 때는 위층 복도의 바닥이 무너지며 무덤으로 떨어집니다. 가짜 문을 열면 돌문이 물러나고 탈출 복도를 걷는 마무리 장면으로 이어집니다. 보물을 챙겼는지에 따라 마지막 장면을 다르게 만들었습니다.
- **사운드:** 발소리·돌문·시대별 배경음은 주로 OpenGameArt의 무료 음원을 사용했습니다. 대부분은 CC0이며, 획득 차임과 이동 효과음 등 CC-BY 3.0 음원은 출처를 따로 밝혔습니다. 원하는 소리가 없을 때는 기존 음원을 가공했습니다. 예를 들어 청동 거울이 돌받침에 걸리는 소리는 돌 부딪는 음원을 4분의 1 속도로 느리게 한 뒤 낮은 합성음을 겹쳐 만들었습니다. 음원 로딩이 끝나지 않았거나 실패했을 때는 브라우저에서 직접 합성한 소리로 대체합니다.

2-5. 상호작용 · 화면 표시(UI)

- **조준 보조:** 작은 물체를 지나치게 정확히 겨누지 않아도 잡을 수 있게 했습니다. 화면 정가운데와 그 주변으로 아홉 개의 광선을 쏘고, 그중 중앙에 가까운 대상을 먼저 고르는 multi-raycast 방식입니다.
- **소지품:** 여러 물건을 지닐 수 있고, 숫자 키로 지금 들 물건을 고릅니다. 시대가 달라져도 집고 내려놓는 조작은 같습니다.
- **화면 안내:** 바라보는 대상에 맞춰 지금 할 수 있는 행동을 표시합니다. 벽돌 당기기처럼 키를 길게 눌러야 하는 행동은 게이지로 남은 시간을 보여 줍니다.

2-6. 최적화 · 품질 관리

- **거울 최적화:** 거울에 상을 비추려면 거울 속 장면을 한 번 더 그려야 합니다. 게임 안의 거울 넷을 모두 같은 품질로 계속 그리면 장면을 최대 다섯 번씩 그리게 됩니다. 그래서 지금 바라보는 거울만 매 프레임 온전한 화질로 처리했고, 멀리 있거나 시야 가장자리에 걸린 거울은 해상도와 갱신 횟수를 낮췄습니다. 그림자 해상도도 조정하고, 첫 클릭 때 필요한 그래픽 자원을 미리 준비해 처음 거울을 볼 때 생기는 멈칫거림을 줄였습니다.
- **오류 진단:** 로딩 중 문제가 생기면 시작 화면에 원인을 표시하도록 했습니다. 또 화면 없이 실행되는 브라우저에서 게임을 켜고 클릭까지 해 보는 검사를 추가했습니다.
- **배포:** 저장소에 변경 내용을 반영하면 웹 버전도 자동으로 배포됩니다. 플레이어는 별도 설치 없이 주소로 접속하면 됩니다.

3. 협업 · 분업 방식

- **기능 단위로 나눔:** 개인별 영역을 고정하지 않고, 그날 필요한 기능을 보고 일을 나눴습니다. 같은 파일을 여러 명이 바꾸는 경우가 많아 커밋 범위와 통합 순서를 중요하게 봤습니다.

- **작은 단위로 커밋:** 변경 내용을 작게 나눠 자주 커밋했습니다. 마지막 문 퍼즐처럼 변경 폭이 큰 작업은 별도 브랜치에서 완성하고 검사한 뒤 합쳤습니다.
- **합치기 전 검사:** 작업을 합칠 때마다 전체 자동 테스트를 실행했습니다. 다른 사람의 작업을 합친 뒤 퍼즐 규칙이 바뀌거나 진행이 막히는 문제를 줄이기 위해서입니다.
- **서로 플레이해 확인:** 한 사람이 만든 기능은 다른 사람이 직접 플레이하거나 값을 재 보며 확인했습니다. 물건이 공중에 떠 보이거나 거울이 기울어진 문제는 플레이 중 발견하자마자 공유하고 고쳤습니다.
- **AI 도구:** Claude Code는 검증 도구 작성, 거울 반사 각도 계산, 버그 원인 추적, 문서 초안 작성에 사용했습니다. 제안된 내용을 그대로 반영하지는 않았고, 게임 규칙과 화면에 맞는지 팀원이 플레이하며 확인한 뒤 수정했습니다. 최종 판단과 결과물 확인도 팀이 직접 맡았습니다.

프로젝트 저장소: <https://github.com/ppco915/mirrorlight> · 플레이: <https://ppco915.github.io/mirrorlight/>